Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»



ПРАКТИЧЕСКАЯ РАБОТА

№ 4

Дисциплина(модуль)/предмет МДК 03.01 Сопровождение и обслуживание программного обеспечения компьютерных систем

Тема: Версионирование API и обратная совместимость

ВАРИАНТ № 3 — Climate Control

Студент (ка)/учащийся Бойцов Даниил Антонович / _____ /
Ф. И. О. подпись

Группа / Класс **И13-24-11**

Руководитель: **Александра Анатольевна Чудинова** /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «05» ноября 2026 г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 КОНТРАКТЫ И МЕХАНИЗМ ВЕРСИОНИРОВАНИЯ 4
2 РЕАЛИЗАЦИЯ И ТЕСТ СТАРОГО КЛИЕНТА 5
3 ФАКТИЧЕСКИЕ РЕЗУЛЬТАТЫ ИСПЫТАНИЙ 6
4 ПОВТОРНАЯ ПРОВЕРКА РЕШЕНИЯ НА WINDOWS 7
ЗАКЛЮЧЕНИЕ 8
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 9
ПРИЛОЖЕНИЕ А ЛИСТИНГИ ИСХОДНОГО КОДА 10
ПРИЛОЖЕНИЕ Б СКРИНШОТЫ 12

ВВЕДЕНИЕ

Практическая работа посвящена теме «Версионирование API и обратная совместимость» для модуля Climate Control АИС «Умный кампус», вариант 3. Система принимает температуру и влажность от IoT-датчиков. Объект рассмотрения — программные средства приёма, хранения и сопровождения телеметрии.

В отчёте представлены решения для своего варианта, ссылки на исходники, результаты программных проверок и ограничения стенда. Листинги помещены в приложении А, действительные скриншоты — в приложении Б. Проект не подключён к физическому отоплению или вентиляции.

Материалы практической работы в репозитории:
https://github.com/daniil179/mdk0301-climate-control/tree/main/docs/pr04

1 КОНТРАКТЫ И МЕХАНИЗМ ВЕРСИОНИРОВАНИЯ

Цель работы — реализовать версии REST API в ASP.NET Core и проверить сохранение работоспособности старого клиента. Для Climate Control выбран явный номер в URL: маршруты читаются однозначно, их можно независимо проверять и выводить из эксплуатации.

Реализация находится в src/Climate.Api/Controllers/ReadingsController.cs. Общие данные хранятся в таблице Telemetry, но проекции ответа разделены. v1 возвращает SensorId, TemperatureC и HumidityPercent. v2 возвращает температуру с единицей Cel, время измерения и признак качества. POST телеметрии остаётся в v1.

Сведения по теме «маршруты и контракты версий api» представлены в таблице 1.

Таблица 1 — Маршруты и контракты версий API

Операция	Маршрут	Контракт
Отправка	POST /api/v1/telemetry	Подписанное измерение; 201 при сохранении.
Старый клиент	GET /api/v1/climate/readings/latest/{sensorId}	Плоский JSON v1, HTTP 200.
Новый клиент	GET /api/v2/climate/readings/latest/{sensorId}	Отдельный DTO v2, HTTP 200.
Неизвестная версия	GET /api/v9/climate/readings/latest/{sensorId}	Маршрут отсутствует, 404.

Источник: составлено автором для Climate Control варианта 3

Требуется HMAC для метода и точного пути запроса. Чтение чужого SensorId запрещено политикой владельца устройства. Подпись GET строится для пустого тела; timestamp и nonce создаются заново на каждый запрос. Ключи тестового реестра предназначены только для изолированного стенда.

2 РЕАЛИЗАЦИЯ И ТЕСТ СТАРОГО КЛИЕНТА

Вместо подключения библиотеки выбора версии используются два явных маршрута ASP.NET Core. Это допустимый механизм URL-версионирования для двух фиксированных контрактов; автоматический выбор версии по Accept-заголовку не реализуется.

Листинг приведён из ReadingsController.cs; класс помечен политикой TelemetryReader. Полная реализация приложена в проекте. Доступ контролируется сервером до выдачи строки. Измерение выбирается по SensorId и убывающему Sequence, поэтому «последнее» означает последнее в последовательности устройства.

Тест V01 запускает настоящий ASP.NET Core pipeline через WebApplicationFactory и TestServer [1], отправляет подписанное измерение, затем читает v1. Старый DTO десериализуется с запретом неизвестных членов; проверяются температура 23,5 и влажность 45. При случайной замене v1 на v2 тест перестанет проходить.

3 ФАКТИЧЕСКИЕ РЕЗУЛЬТАТЫ ИСПЫТАНИЙ

Сведения по теме «результаты контрактных проверок» представлены в таблице 2.

Таблица 2 — Результаты контрактных проверок

Тест	Ожидаемый результат	Факт
V01	Старый DTO читает v1; значения сохранены.	PASS
V02	v2 содержит temperature.unit = Cel.	PASS
V03	Без HMAC чтение запрещено, 401.	PASS
V04	Чужой SensorId запрещён, 403.	PASS
V05	Неизвестная версия не маршрутизируется, 404.	PASS
V06	Событие из SQLite outbox доставлено через подписанный API; в БД одна строка.	PASS

Источник: составлено автором для Climate Control варианта 3

Реально выполнено 26 интеграционных сценариев: шесть новых проверок и двадцать ранее подготовленных проверок валидации и защиты. Ошибок и пропусков нет. Дополнительно выполнено 60 модульных сценариев, также без ошибок. Файлы unit-results.json, integration-results.json и текстовые журналы приложены.

Целостность запроса проверяется HMACSHA256 [2]. Формат ответов соответствует JSON [3]. Полные примеры исходного запуска сохранены в evidence/original/v1-response.json и v2-response.json.

4 ПОВТОРНАЯ ПРОВЕРКА РЕШЕНИЯ НА WINDOWS

После дополнения портфолио выполнены стандартные команды dotnet build и dotnet test: 63 модульных и 26 интеграционных тестов прошли, ошибок и пропусков нет. Тесты V01–V06 проверяют v1/v2, старый клиент и отрицательные сценарии. Старые JSON-примеры выше сохранены как результаты исходного запуска; свежие журналы доступны в evidence/final. Контроллер и фрагмент журнала показаны на рисунках Б.1 и Б.2.

ЗАКЛЮЧЕНИЕ

Реализованы независимые URL-контракты v1/v2. Старый клиент продолжает читать v1 после добавления новой структуры. Негативные сценарии подтверждают отказ без подписи, при чужом устройстве и неизвестной версии. Полная программная проверка завершена успешно.

Материалы относятся к модулю Climate Control варианта 3. Проверенный программный стенд не подключён к физическому отоплению или вентиляции кампуса. Промышленная приёмка оборудования в рамках работы не проводилась.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Microsoft Learn. Integration tests in ASP.NET Core. — URL: https://learn.microsoft.com/en-us/aspnet/core/test/integration-tests?view=aspnetcore-8.0 (дата обращения: 11.11.2026).
2. Microsoft. HMACSHA256 API documentation. — URL: https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.hmacsha256 (дата обращения: 11.11.2026).
3. IETF. RFC 8259. The JavaScript Object Notation Data Interchange Format. — URL: https://www.rfc-editor.org/rfc/rfc8259 (дата обращения: 11.11.2026).

ПРИЛОЖЕНИЕ А ЛИСТИНГИ ИСХОДНОГО КОДА

Ниже приведены фрагменты реальных исходников. Для сборки используется всё решение, размещённое в общем репозитории. Полные файлы доступны по ссылкам; номера строк относятся к опубликованной версии.

Общее решение: https://github.com/daniil179/mdk0301-climate-control/tree/main/projects/ClimateControl

src/Climate.Api/Controllers/ReadingsController.cs

tests/Climate.IntegrationTests/VersioningTests.cs

Листинг А.1 — Ответ старого контракта v1

Исходный файл: ReadingsController.cs

```
   [HttpGet("api/v1/climate/readings/latest/{sensorId}")]
   public async Task<IActionResult> GetV1(string sensorId, CancellationToken
ct)
   {
      if (sensorId != User.FindFirstValue(ClaimTypes.NameIdentifier)) return
Forbid();
      var row = await db.Telemetry.Where(x => x.SensorId == sensorId)
         .OrderByDescending(x => x.Sequence).FirstOrDefaultAsync(ct);
      return row is null ? NotFound() : Ok(new
      {
         sensorId = row.SensorId, temperatureC = row.TemperatureC,
         humidityPercent = row.HumidityPercent
      });
   }
```

ПРИЛОЖЕНИЕ Б СКРИНШОТЫ

На рисунке Б.1 показан фрагмент: Реализация контракта v1 в GitHub.

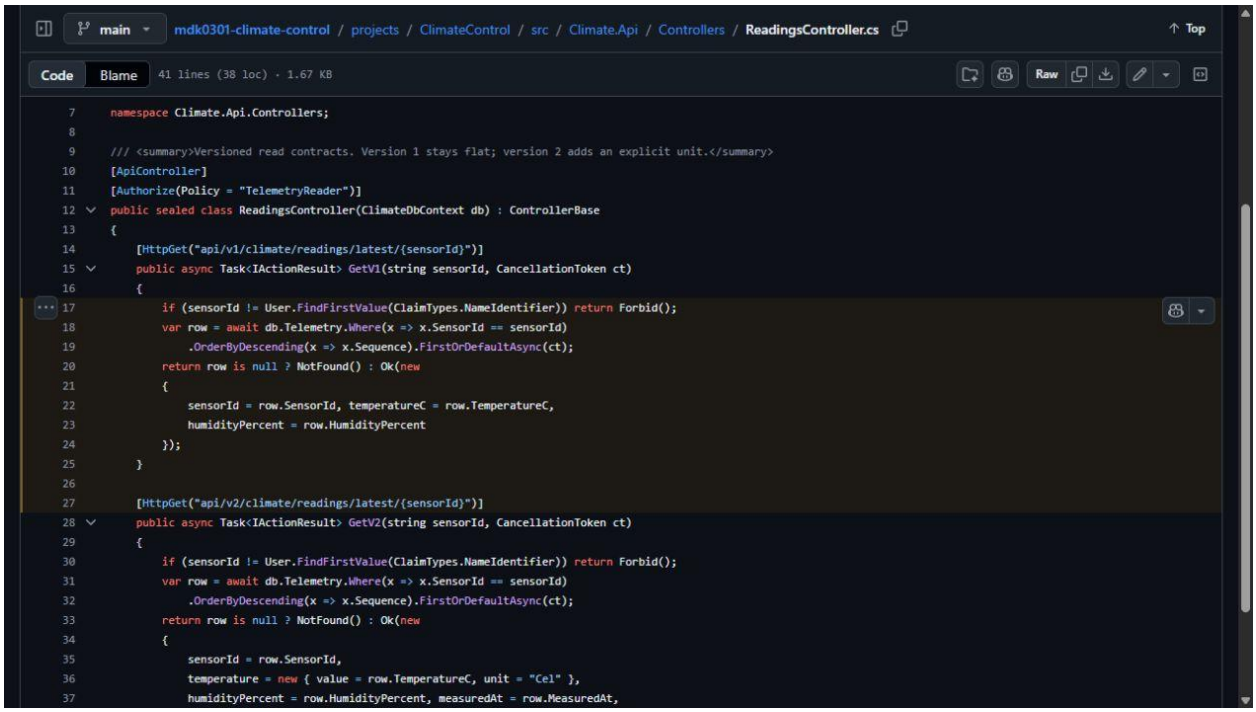


Рисунок Б.1 — Реализация контракта v1 в GitHub

На рисунке Б.2 показан фрагмент: Фрагмент журнала исходных 60 модульных и 26 HTTP тестов в Visual Studio Code.

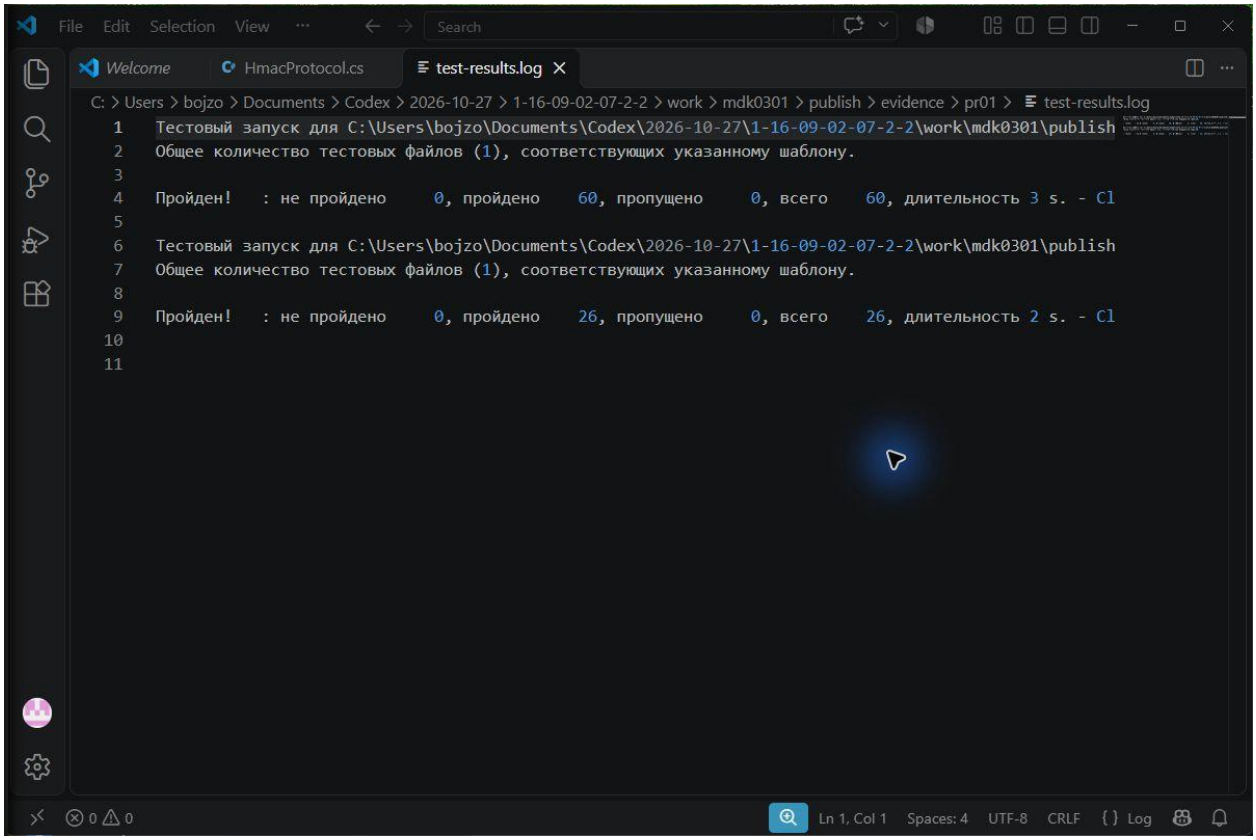


Рисунок Б.2 — Фрагмент журнала исходных 60 модульных и 26 HTTP тестов в Visual Studio Code